

The Wazuh Playbook

A complete, detailed guide to Wazuh — from agent deployment
to custom detection rules and active response

*From a single enrolled agent to a tuned detection pipeline, this
playbook builds the architecture knowledge and rule-writing fluency
that SOC analysts running Wazuh rely on every day.*

Author: Swastik Sagar
Final-year Computer Science Undergraduate
SOC Analyst Intern

August 5, 2026

Preface

Wazuh is the leading open-source SIEM/XDR platform, and for a good reason: it combines log analysis, file integrity monitoring, vulnerability detection, configuration assessment, and active response in one free, self-hostable stack — everything a smaller SOC or a home lab needs to build real detection engineering skill without a commercial license.

This playbook goes well beyond a general overview. It covers Wazuh’s architecture in depth, a full deployment walkthrough, and then spends real time on the two things that separate a default install from a useful detection platform: decoders (how raw logs become structured fields) and rules (how those fields become alerts). File Integrity Monitoring, vulnerability detection, active response, and malware detection each get their own chapter, because each is a genuinely different capability with its own configuration model — not just a checkbox to enable.

What this book covers

- Wazuh architecture: agent, manager, indexer, dashboard, and how they interact
- Installation and deployment, single-node and distributed
- Agent management: enrollment, groups, and configuration
- Writing custom decoders to parse raw logs into structured fields
- Writing and tuning custom rules, including multi-stage correlation
- File Integrity Monitoring (FIM) configuration and use
- Vulnerability detection and Security Configuration Assessment (SCA)
- Active response: automated reactions to alerts
- Log data collection from files, Windows, and syslog sources
- Malware detection with YARA integration and VirusTotal
- Dashboards, the Wazuh API, and third-party integrations
- Practical detection recipes for common scenarios

Swastik Sagar

Contents

Preface	i
1 Introduction to Wazuh	1
1.1 What Wazuh is	1
1.2 Core capabilities	1
2 Architecture in Depth	2
2.1 The four components	2
2.2 The manager's internal pipeline	2
2.3 Single-node vs. distributed deployment	2
3 Installation and Deployment	3
3.1 Quick installation	3
3.2 Installing an agent	3
3.3 Verifying the install	3
4 Agent Management	4
4.1 Enrollment	4
4.2 Agent groups	4
4.3 Checking agent status	4
5 Decoders	5
5.1 What a decoder does	5
5.2 Decoder anatomy	5
5.3 Useful decoder fields	5
5.4 Testing a decoder	5
6 Rules Deep Dive	7
6.1 How rules match	7
6.2 A basic custom rule	7
6.3 Multi-stage correlation	7
6.4 Rule levels	8
6.5 Rule options worth knowing	8

7	File Integrity Monitoring	9
7.1	What FIM tracks	9
7.2	Configuring FIM	9
7.3	What check_all actually checks	9
8	Vulnerability Detection and SCA	10
8.1	Vulnerability detection	10
8.2	Security Configuration Assessment (SCA)	10
9	Active Response	11
9.1	What active response does	11
9.2	Configuring an active response	11
10	Log Data Collection	12
10.1	Log collection sources	12
10.2	Log rotation and multiline logs	12
11	Malware Detection	13
11.1	Rootcheck and signature-based detection	13
11.2	YARA integration	13
11.3	VirusTotal integration	13
12	Dashboards, API, and Integrations	15
12.1	The Wazuh dashboard	15
12.2	The Wazuh API	15
12.3	Common third-party integrations	15
13	Practical Detection Recipes	16
13.1	Detecting a brute-force SSH attack	16
13.2	Alerting on a new local admin account	16
13.3	Watching for FIM changes to a critical config file	16
13.4	Correlating a vulnerability with active exploitation attempts	16

Introduction to Wazuh

1.1 What Wazuh is

Wazuh is a free, open-source security platform providing unified XDR (Extended Detection and Response) and SIEM capabilities. Built on a fork of OSSEC's agent technology and paired with OpenSearch for storage and search, it collects, analyzes, and correlates security data from endpoints, cloud services, and network devices.

1.2 Core capabilities

- **Log data analysis** — collecting and analyzing logs from agents and remote syslog sources
- **File Integrity Monitoring (FIM)** — detecting changes to monitored files and registry keys
- **Vulnerability detection** — correlating installed software against known CVE feeds
- **Security Configuration Assessment (SCA)** — checking systems against hardening benchmarks like CIS
- **Malware and rootkit detection** — signature and behavior-based detection, plus YARA integration
- **Active response** — automated reactions to alerts, such as blocking an IP
- **Cloud security monitoring** — native modules for AWS, Azure, and GCP audit data

Why it matters for a home lab

Every capability in this book runs entirely on infrastructure you control, with no licensing cost — making Wazuh one of the few platforms where you can genuinely practice detection engineering end to end, including intentionally triggering and tuning real detections, without any usage restrictions.

Architecture in Depth

2.1 The four components

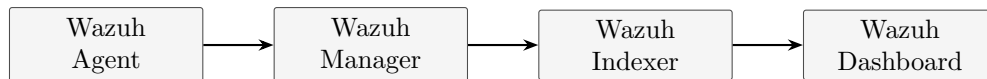


Figure 2.1. Data flows one direction through the stack: agents collect, the manager analyzes, the indexer stores, the dashboard visualizes.

Component	Role
Agent	Lightweight process on a monitored endpoint; collects logs, FIM data, runs SCA and vulnerability checks
Manager	Receives agent data, applies decoders and rules, generates alerts, manages agent configuration
Indexer	OpenSearch-based backend that stores and indexes alert and archive data
Dashboard	Web UI (based on OpenSearch Dashboards) for browsing, searching, and visualizing alerts

2.2 The manager's internal pipeline



Figure 2.2. Every single event the manager receives passes through this exact sequence – decoders always run before rules can match.

2.3 Single-node vs. distributed deployment

Deployment	When to use it
Single-node (all-in-one)	Labs, small environments; manager, indexer, and dashboard on one host
Distributed	Production at scale; indexer cluster separate from manager cluster for resilience and performance

Installation and Deployment

3.1 Quick installation

The all-in-one installation script

```
curl -s0 https://packages.wazuh.com/4.7/wazuh-install.sh
sudo bash wazuh-install.sh -a
```

The `-a` flag installs all components (indexer, manager, dashboard) on a single node — the standard starting point for a lab or small deployment.

3.2 Installing an agent

Linux agent, Debian/Ubuntu

```
curl -so wazuh-agent.deb \
https://packages.wazuh.com/4.x/wazuh-agent.deb
sudo WAZUH_MANAGER='manager-ip' dpkg -i wazuh-agent.deb
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

Windows agent, PowerShell

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent.msi \
-OutFile wazuh-agent.msi
msiexec.exe /i wazuh-agent.msi /q WAZUH_MANAGER='manager-ip'
NET START WazuhSvc
```

3.3 Verifying the install

On the manager

```
sudo systemctl status wazuh-manager
sudo /var/ossec/bin/wazuh-control status
```

Certificates for the indexer

The indexer requires TLS certificates for node-to-node and client communication — the install script's certificate generation tool (`wazuh-certs-tool.sh`) handles this automatically for a single-node setup, but is worth understanding before attempting a distributed deployment where certificates must match multiple nodes.

Agent Management

4.1 Enrollment

An agent enrolls with the manager once (registering its key), then connects continuously afterward using that key for authentication — enrollment can happen automatically at install time, or manually via the manager’s agent management tools.

Manual agent management on the manager

```
sudo /var/ossec/bin/manage_agents
sudo /var/ossec/bin/agent_control -l
sudo /var/ossec/bin/agent_control -i <agent-id>
```

4.2 Agent groups

Agents can be organized into **groups**, each with its own shared configuration (`agent.conf`) — letting you apply different FIM paths, log sources, or SCA policies to, say, web servers versus workstations, without maintaining per-agent configuration by hand.

Assigning an agent to a group

```
sudo /var/ossec/bin/agent_groups -a -i <agent-id> -g webserver
```

4.3 Checking agent status

Status	Meaning
Active	Agent connected and sending data normally
Disconnected	Agent hasn’t checked in within the expected interval
Never connected	Registered but has never successfully connected
Pending	Registered, waiting for the agent to complete enrollment

Disconnected agents are a detection gap

A silently disconnected agent is a monitoring blind spot that’s easy to miss — alerting on agent disconnection itself (Wazuh ships a rule for exactly this) is one of the highest-value, lowest-effort detections to enable in any deployment.

Decoders

5.1 What a decoder does

A decoder takes a raw, unstructured log line and extracts it into named fields — source IP, username, status, and so on — that rules can then match against precisely, instead of doing fragile matching against raw text.

5.2 Decoder anatomy

A basic decoder, in `local.decoder.xml`

```
<decoder name="myapp-login">
  <program_name>myapp</program_name>
</decoder>
<decoder name="myapp-login">
  <parent>myapp-login</parent>
  <regex>user (\S+) login (\w+) from (\S+)</regex>
  <order>user, status, srcip</order>
</decoder>
```

The first decoder matches on the syslog program name to identify which logs it applies to; the second (a child decoder) extracts the actual fields using a regular expression, with **order** mapping each capture group to a field name.

5.3 Useful decoder fields

Field	Purpose
<code>program_name</code>	Matches the syslog program/process name
<code>prematch</code>	An initial pattern that must match before deeper parsing runs
<code>regex</code>	The extraction pattern, with capture groups
<code>order</code>	Maps capture groups, in order, to field names
<code>fts</code>	Marks fields used for first-time-seen alerting

5.4 Testing a decoder

wazuh-logtest

```
sudo /var/ossec/bin/wazuh-logtest
```

Paste a sample raw log line in and **wazuh-logtest** shows exactly which decoder matched and what fields it extracted — always verify a new decoder here before trusting it in production, since a silently non-matching decoder produces no error, just no extracted fields.

Rules Deep Dive

6.1 How rules match

Rules match against decoded fields (or raw log text), and can reference other rules to build multi-stage correlation logic — a rule can require that a specific earlier rule already matched, within a time window, before it fires itself.

6.2 A basic custom rule

local_rules.xml

```
<group name="myapp,">
<rule id="100010" level="5">
<decoded_as>myapp-login</decoded_as>
<field name="status">failed</field>
<description>myapp failed login</description>
</rule>
</group>
```

6.3 Multi-stage correlation

A brute-force rule built on the rule above

```
<rule id="100011" level="10" frequency="8"
timeframe="120">
<if_matched_sid>100010</if_matched_sid>
<same_source_ip/>
<description>Possible brute force against myapp</description>
<mitre>
<id>T1110</id>
</mitre>
</rule>
```

`frequency` and `timeframe` require the referenced rule to match that many times within that many seconds; `same_source_ip` narrows the correlation to repeated attempts from one source specifically.

6.4 Rule levels

Level	Typical meaning
0–3	Informational, no action needed
4–7	Low-to-medium, worth a glance
8–11	Notable security event, should be reviewed
12–15	High severity, often correlation-based
16	Maximum severity, reserved for the most critical rules

6.5 Rule options worth knowing

Option	Purpose
<code>if.sid</code>	Requires a specific parent rule ID to have matched first
<code>if.group</code>	Requires the event to belong to a specific decoder/rule group
<code>same.source.ip</code>	Correlation must come from the same source IP
<code>options</code>	<code>no.full.log</code> suppresses storing the raw log; <code>alert.by.email</code> sends email directly
<code>mitre</code>	Tags the rule with a MITRE ATT&CK technique ID

Test before deploying

Always run new or modified rules through `wazuh-logtest` against both a matching and a non-matching sample log — a rule that’s too broad generates noise, and one that’s too narrow silently never fires, and neither failure mode produces an obvious error.

File Integrity Monitoring

7.1 What FIM tracks

FIM monitors specified files and directories for creation, modification, permission changes, and deletion — generating an alert with a diff of exactly what changed, which attribute changed it, and (on supported systems) which process was responsible.

7.2 Configuring FIM

ossec.conf: basic FIM configuration

```
<syscheck>
<directories check_all="yes" realtime="yes">/etc,/usr/bin,/usr/sbin</directories>
<directories check_all="yes">/home</directories>
<ignore>/etc/mtab</ignore>
<frequency>43200</frequency>
</syscheck>
```

`realtime="yes"` uses inotify (Linux) or equivalent OS hooks for immediate detection rather than waiting for the next scheduled scan; `frequency` sets the full scheduled scan interval in seconds.

7.3 What `check_all` actually checks

Attribute	Tracked
Size	File size changes
Permissions	Mode/ACL changes
Owner	UID/GID changes
Hash	MD5/SHA1/SHA256 content changes
mtime	Modification timestamp changes

Realtime vs. scheduled trade-off

Realtime monitoring on a huge, frequently-changing directory (like a build server's output folder) can generate significant event volume and resource usage — scope realtime monitoring to genuinely sensitive paths (system binaries, configuration, web roots) and leave less critical paths on the scheduled scan instead.

Vulnerability Detection and SCA

8.1 Vulnerability detection

Wazuh's vulnerability detector correlates each agent's installed software inventory against CVE feeds (NVD and vendor-specific feeds), generating an alert for every match — no active scanning of the target is required, since it works entirely from the software inventory the agent already collects.

Enabling the vulnerability detector, `ossec.conf` on the manager

```
<vulnerability-detector>
<enabled>yes</enabled>
<index-status>yes</index-status>
<feed-update-interval>60m</feed-update-interval>
</vulnerability-detector>
```

8.2 Security Configuration Assessment (SCA)

SCA runs policy checks against a host's actual configuration — comparing it to a hardening benchmark like CIS — and reports a pass/fail per check, along with specific remediation guidance for each failure.

Enabling SCA on an agent, `ossec.conf`

```
<sca>
<enabled>yes</enabled>
<scan_on_start>yes</scan_on_start>
<interval>12h</interval>
</sca>
```

Vulnerability detection vs. SCA, in one sentence

Vulnerability detection asks "is any installed software known to be exploitable"; SCA asks "is this system configured according to a hardening standard" — they're complementary, not overlapping, and a mature deployment runs both.

Active Response

9.1 What active response does

Active response executes a script automatically when a matching alert fires — most commonly blocking an IP address, but capable of running any script the deployment defines, from disabling an account to killing a process.

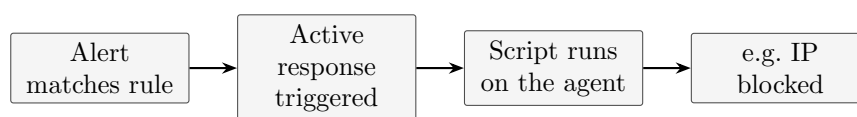


Figure 9.1. *Active response closes the loop from detection to automated containment, without waiting on an analyst.*

9.2 Configuring an active response

ossec.conf on the manager

```

<command>
<name>firewall-drop</name>
<executable>firewall-drop</executable>
<timeout_allowed>yes</timeout_allowed>
</command>
<active-response>
<command>firewall-drop</command>
<location>local</location>
<rules_id>100011</rules_id>
<timeout>600</timeout>
</active-response>
  
```

`timeout` automatically reverses the response (e.g. unblocking the IP) after the specified number of seconds, preventing a permanent self-inflicted lockout from a false positive.

Start conservative

Active response is powerful and genuinely risky — an overly broad rule tied to an automated block can cause a real outage. Start by tying active response only to high-confidence, well-tuned rules, and always configure a timeout rather than a permanent action.

Log Data Collection

10.1 Log collection sources

Source type	Example
Local files	Application logs, custom log files via a path pattern
Windows Event Log	Security, System, Application, and custom channels
Syslog (remote)	Network devices, firewalls, anything forwarding via UDP/TCP syslog
Command output	Periodic output of a specified command, treated as a log
Journald	systemd's journal, on modern Linux distributions

ossec.conf: monitoring a log file

```
<localfile>
<log_format>syslog</log_format>
<location>/var/log/auth.log</location>
</localfile>
```

ossec.conf: Windows Event Log

```
<localfile>
<log_format>eventchannel</log_format>
<location>Security</location>
</localfile>
```

ossec.conf: remote syslog listener, on the manager

```
<remote>
<connection>syslog</connection>
<port>514</port>
<protocol>udp</protocol>
<allowed-ips>192.168.1.0/24</allowed-ips>
</remote>
```

10.2 Log rotation and multiline logs

Wazuh automatically follows log rotation (detecting when a monitored file has been rotated and picking up the new file). Multi-line log entries need a matching `log_format` or a custom decoder with an appropriate `prematch`, since the default behavior treats each line as a separate event.

Malware Detection

11.1 Rootcheck and signature-based detection

Wazuh's built-in rootcheck module scans for known rootkit signatures, suspicious file permissions, hidden processes, and other system-level anomalies on a scheduled basis — a baseline capability enabled by default on most installations.

11.2 YARA integration

YARA rules describe patterns (byte sequences, strings, structural traits) characteristic of known malware families. Wazuh doesn't run YARA natively, but the standard pattern uses FIM to trigger a YARA scan automatically the moment a new or modified file appears in a monitored directory.

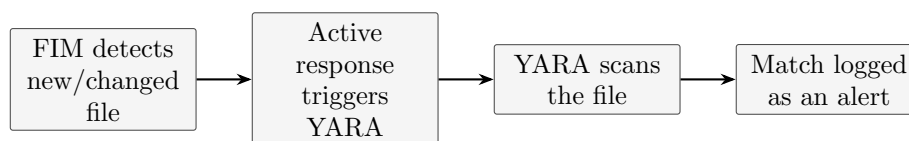


Figure 11.1. *The standard Wazuh + YARA pattern: FIM provides the trigger, active response runs the scan, and the result flows back in as a normal log event.*

The active response script calling YARA, conceptually

```
#!/bin/bash
FILENAME=$(cat | grep -oP '(?<="path":")[^"]+')
yara -r /var/ossec/etc/rules/yara_rules.yar \
"$FILENAME" >>/var/log/yara_scan.log
```

11.3 VirusTotal integration

Wazuh can automatically submit file hashes triggered by FIM events to the VirusTotal API and log the result — useful as a fast first-pass triage signal, though it depends on the file already being known to VirusTotal's corpus and is rate-limited on the free API tier.

Enabling VirusTotal integration, ossec.conf on the manager

```
<integration>
<name>virustotal</name>
<api_key>your_api_key_here</api_key>
<rule_id>550,554</rule_id>
<alert_format>json</alert_format>
```

</integration>

Free-tier rate limits

The free VirusTotal API tier allows a small number of requests per minute — fine for a lab, but a busy FIM configuration on a production fleet can exceed it quickly, so scope which rule IDs trigger a lookup carefully rather than submitting every FIM event.

Dashboards, API, and Integrations

12.1 The Wazuh dashboard

Built on OpenSearch Dashboards, the Wazuh dashboard ships with prebuilt views for security events, FIM, vulnerabilities, SCA results, and MITRE ATT&CK coverage — and supports the same custom visualization and dashboard-building tools OpenSearch/Kibana users are already familiar with.

12.2 The Wazuh API

The REST API exposes agent management, rule/decoder management, and querying capabilities programmatically — the same actions available in the dashboard, scriptable for automation.

Authenticating and querying via the API

```
TOKEN=$(curl -u wazuh:wazuh -k -X POST \
  "https://localhost:55000/security/user/authenticate" | jq -r .data.token)
curl -k -X GET "https://localhost:55000/agents?status=active" \
  -H "Authorization: Bearer $TOKEN"
```

12.3 Common third-party integrations

Integration	Purpose
Slack / email / webhook	Route alerts to a team channel or ticketing system
TheHive / Shuffle	SOAR-style case management and automated playbooks
Suricata	Ingest network IDS alerts alongside host-based detections
Cloud provider modules	Native AWS/Azure/GCP audit log collection

Practical Detection Recipes

13.1 Detecting a brute-force SSH attack

This ships as a default rule, but the logic is worth knowing

```
<rule id="5720" level="10" frequency="8"
timeframe="120">
<if_matched_sid>5716</if_matched_sid>
<same_source_ip/>
<description>Multiple SSH authentication failures</description>
</rule>
```

13.2 Alerting on a new local admin account

Windows, watching Event ID 4720/4732

```
<rule id="100020" level="10">
<if_sid>60106</if_sid>
<field name="win.system.eventID">4732</field>
<description>User added to a privileged local group</description>
<mitre>
<id>T1136</id>
</mitre>
</rule>
```

13.3 Watching for FIM changes to a critical config file

ossec.conf, scoped realtime FIM

```
<syscheck>
<directories check_all="yes" realtime="yes">/etc/ssh/sshd_config</directories>
</syscheck>
```

Combined with the default FIM rules, any modification generates an alert automatically — no additional custom rule required for the basic case.

13.4 Correlating a vulnerability with active exploitation attempts

- Vulnerability detector flags a host running a known-vulnerable service version
- Cross-reference Suricata/network IDS alerts targeting that same host and port

- Prioritize triage for hosts where both signals are present — a known-vulnerable, actively-targeted host is a materially higher priority than either signal alone

Start from the default ruleset

Wazuh ships with several thousand default rules covering common log sources out of the box — before writing custom rules, check whether the default ruleset (`/var/ossec/ruleset/rules/`) already covers the use case, and extend or tune from there rather than starting from scratch.